



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

**SCHOOL OF OPEN, DISTANCE AND
eLEARNING
IN COLLABORATION WITH**

**INSTITUTE OF COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY**

**DEPARTMENT OF INFORMATION
TECHNOLOGY**

ICS 2104: OBJECT-ORIENTED PROGRAMMING I

P.O. Box 62000, 00200

Nairobi, Kenya

ICS 2104: OBJECT-ORIENTED PROGRAMMING I

Course Description

Introduction to Object Oriented Programming. Abstraction (data, control) and abstract data types. Objects and instances, Classes and hierarchy via composition and derivation (inheritance). Early and late binding, The concepts of Information Hiding and access modifiers, Encapsulation and Polymorphism. Constructors and Destructor. Friend functions. Inline functions, Virtual functions. Operator overloading and function overloading. Files and Database connectivity. Concepts demonstrated using C++.

Prerequisite

BIT 2104 Introduction to Programming and Algorithms

Course Purpose

This course is intended to highlight the abstract notion of object-oriented programming and code reuse

Learning Outcomes

Upon completion of this course, you should be able to:

- Demonstrate an understanding of the principles of OOP
- Illustrate good object-oriented programming skills
- Differentiate the capabilities and limitations of Java/C++
- Develop a two tier application that communicate with a database

Instruction methodology

computers, textbooks, appropriate software

Assessment information

The module will be assessed as follows;

- 10% of marks from two (2) assignments to be submitted online

-
- 20% of marks from one written CAT to be administered at JKUAT main campus or one of the approved centres
 - 70% of marks from written Examination to be administered at JKUAT main campus or one of the approved centres

Course Textbooks

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi

Reference Textbooks

- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

Course Journals

- JOOP - Journal of Object-oriented Programming Publications.
- Journal: Object Oriented Systems, 2000.

Reference Journals

- Journal: ACM Computing Surveys - CSUR , vol. 43, no. 3 2011.
- Journal of Programming Language.

Contents

1	Introduction Object-Oriented Programs	1
1.1	Introduction to object-oriented programs	2
1.2	Fundamental Object-Oriented Concepts	2
1.3	Writing an Object-Oriented Program using a High-level Language .	4
2	The Basics of an Object-Oriented Program Written in C++	7
2.1	Introduction	7
2.2	The basics of a C++ Program	7
2.3	Testing	11
2.4	Input and Output Operations	12
3	Classes and Objects	14
3.1	Introduction to Classes	14
3.2	Declaring a class	15
3.3	Defining Member Functions	16
3.4	Creating objects from a class	17
3.5	Accessing class members	17
4	Friend Functions, Static Functions and Constant Functions	21
4.1	A Friend Function	21
4.1.1	Making an External Function a Friend Function	21
4.1.2	Making a Member Function a Friend Function	23
4.1.3	Making all Member Functions to be Friends	24
4.2	const member function	25
4.3	static member function	25
5	Constructors and Destructors	28
5.1	Constructors	28

5.2	Declaration of Constructors	28
5.3	Calling Constructors	29
5.4	Types of Constructors	29
5.4.1	A Default Constructor	30
5.4.2	Parameterized constructors	30
5.4.3	Copy constructors	31
5.5	Destructors	31
6	Storage classes	34
6.1	Introduction	34
6.2	Auto storage specifier	34
6.3	The register storage class specifier	35
6.4	The static storage class specifier	35
6.5	The extern storage class specifier	36
7	Inheritance	38
7.1	Introduction to inheritance	38
7.2	Redefining and Overloading Member Functions of the Base Class .	41
7.3	Constructors of Derived and Base classes	41
7.4	Virtual Functions and Pure Virtual Functions	41
8	Operator Overloading	45
8.1	Operator Overloading	45
8.2	Operation Overloading Function	45
8.3	Overloading Binary Operators	46
8.3.1	Overloading + operator using a member function	46
8.3.2	Overloading + operator using a non-member function	48
8.4	Overloading Unary Operators	50
8.4.1	Overloading pre-increment/decrement operators using mem- ber functions	50
8.5	Overloading pre-increment/decrement operators using a non-member function	52
8.6	Overloading post-increment/decrement operators using member func- tions	53

8.7	Overloading post-increment/decrement operators using non-member function	55
8.8	Overloading stream insertion (<<) and extraction (>>) operators . .	57
8.8.1	Overloading the stream insertion (<<) operators	57
9	Template	62
9.1	Introduction to Templates	62
9.2	Function Template	62
9.3	Class Template	65
10	Files and Streams	69
10.1	Introduction	69
10.2	Stream	69
10.3	opening a file using constructor function.	70
10.4	Using open() function	71
10.5	Checking for successful opening	71
10.6	Closing a file	72
10.7	Input from and output to a file	72
10.8	Detecting end of a file	72
10.9	File pointers and function for manipulating file pointers	72
	Solutions to Exercises	75

LESSON 1

Introduction Object-Oriented Programs

Learning outcomes

Upon completing this topic you should be able to:

- Define object-oriented concepts that includes classes, objects, encapsulation, message passing encapsulation, polymorphism and inheritance
- Differentiate an object-oriented programs and an object-oriented programming language
- Describe properties of object-oriented programs

1.1. Introduction to object-oriented programs

Object-oriented programs(OOPs) are programs that are made of objects.

These objects interact with one another to achieve specific objectives.

Objects interact with one another through message passing.

The objects make up an OOP are instantiated from classes.

Classes are building blocks of OOPs. As compared to structured programs, object-oriented programs are easier to develop and maintain.

Object oriented programming is the process of developing an object-oriented program.

The programming languages that are used in writing object-oriented programs are referred to as object-oriented programming languages

1.2. Fundamental Object-Oriented Concepts

Fundamental object-oriented concepts includes:- objects, classes, inheritance, encapsulation, message passing, information hiding, abstraction and polymorphism.

- Object

An object is:

- a) A run-time entity and it consume computer memory
- b) An instance of a class.
- c) An entity that has state, behavior and identity.

- Class

A class is

- a) A template that is used to construct objects.
- b) A collection of objects that have similar state and behavior.
- c) A collection of objects of similar type.

- Inheritance

Inheritance is the ability to define new classes from already existing classes. This is a powerful concept in programming. It facilitates development of programs that are of high quality.

A class that is defined, created or derived from another class is referred to as a sub-class, child class or a derived class.

A class that is used to create or derive another class is referred to as a super-class, base or a parent-class.

There are four types of inheritance:

- a) single inheritance: A sub-class has one parent class
- b) multiple inheritance: A sub-class has more than one parent class. There are some languages such as Java that does not support multiple inheritance.
- c) mult-level inheritance: A sub-class has one parent class and the parent class is a sub-class of another parent class
- d) hybrid inheritance: It is a combination of single-inheritance, multiple inheritance and mult-level inheritance.

A class that is derived from another class inherits properties of the base class.

There are some languages e.g. Java that does not support m

- Encapsulation

Encapsulation is:

- a) enclosing state and behavior of an object.
- b) putting together state and behavior of objects in single unit.

Encapsulation is implemented through the use of classes.

- Information hiding

Information hiding is prevention of state and behavior of an object from direct access by other object. Information hiding is implemented through the use of access specifiers that includes: private, public and protected. private, public and protected are keywords

- Abstraction

Abstraction is the act of showing only the general properties of an object leaving out the detailed properties. Abstraction useful when designing complex programs. A class is an example of an abstract data type(ADT)

- Message passing

Message passing is the way objects interact with one another in an object-oriented program. Objects interact by sending messages to one another. For example, if “person” and “window” are two objects in an object-oriented program, the “person” object may send a message to the “window” requesting it to close.

A message is a request for something to be done. Message passing has the following information.

- a) The name of the object that the message is being sent to
- b) The name of the message
- c) Information that the receiver of the message requires in order to respond to the message.

- Polymorphism

Polymorphism is the ability of two or more objects to respond differently when they are sent the same message. For example, the window and the door objects can be sent close message. However, the way the window object will respond to that message will be different from the way the door object will respond to the same message.

1.3. Writing an Object-Oriented Program using a High-level Language

The following are steps that are followed in order to execute an object-oriented program written in a high-level language

1. First, a text-editor is used to create a program in a high-level language such as Java or C++. The program is written following syntax and semantics of the language being used. This program is referred to as source code. The program source code is saved.
2. The source-code is compiled using a compiler. The compiler checks the source code for syntax errors, and if no error is found, it translate the program into equivalent machine code.
3. Linker combine machine code produced by the compiler with machine code from the libraries included in the program to produce an executable file.
4. The executable file is loaded into the main memory for execution. This is done by a program referred to as loader.

Revision Questions

EXERCISE 1.  Differentiate an object and a class

Example . Define a compiler

Solution: A compiler is a program that checks for syntax errors and translate program source code into machine code

□

EXERCISE 2.  Compare and contrast an object-oriented program and a structured program

EXERCISE 3.  Explain how inheritance improve quality of a program

EXERCISE 4.  Explain the importance of polymorphism and encapsulation

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 2

The Basics of an Object-Oriented Program Written in C++

Learning outcomes

Upon completing this topic you should be able to:

- Write basic C++ programs
- Explain the importance of comments
- List main sections of a C++ program
- Differentiate between user-defined and programmers' defined classes and functions
- Explain why software are tested.

2.1. Introduction

C++ programs consists of functions and classes.

Some of these functions and classes are predefined while others are programmers' defined.

It is advisable to use as many pre-defined classes and functions as possible in a program.

Programs that uses pre-defined functions and classes are of better quality than those that are made of programmers's defined classes and functions. Pre-defined functions and classes are stored in header files.

All C++ programs have main() method. This is the first method that is called by the operating system when the program starting executing.

C++ programming language is a case sensitive language, and therefore the main() function must be written in lower case.

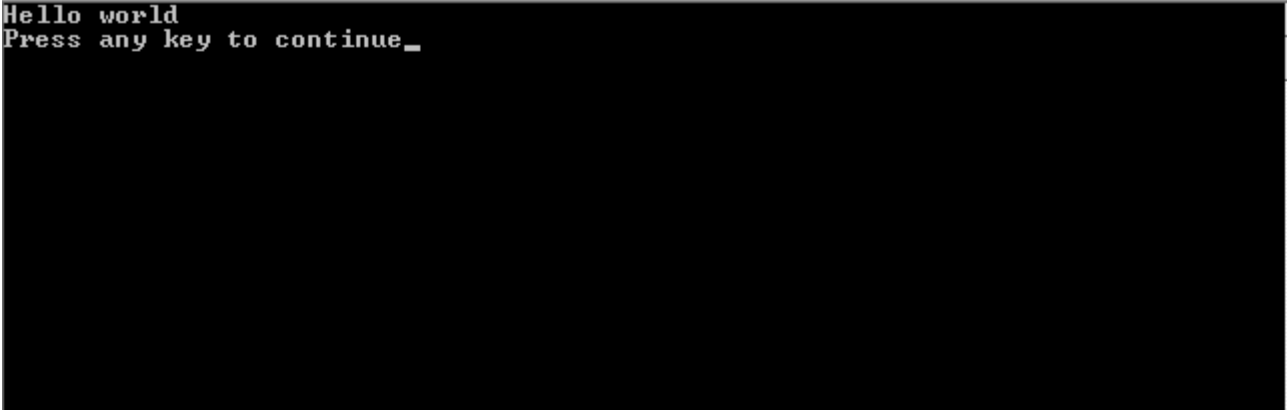
2.2. The basics of a C++ Program

Below are examples of C++ programs.

Example 1: An example of a program that has a function

```
/*=====
```

File name: First.cpp



```
Hello world
Press any key to continue_
```

Table 2.1: Output from the above program

Description: This program demonstrates the minimum number and type of statements necessary to write a simple C++ program.

```
===== */
#include <iostream.h>
void print(void);
int main(void)
{
    print();
    return 0;
}
void print(void){
    cout<<"Hello world" <<endl;
}
```

- Comments

The above program begins with a comment block.

Comments are used to explain what the program source code is doing.

They make it easier for other programmers especially those that are in charge of maintaining the program to understand it better.

In C++, comments begins with `/*` notations and ends with `*/` notations. Everything

between these notations is ignored by the compiler. It is advisable to comment every line of source code.

```
/*This is my first C++ program */
```

One can also use `//(two forward slashes)` notations to comment a line. In that case everything after these notations is ignored by the compiler.

Example:

```
int speed;//store speed of the car
```

- Header Files

The next statement in the program is: `#include<iostream.h>` This statement is known as a pre-processor directive. It instructs the pre-processor to retrieve the code stored in `iostream.h` file and insert it at the point of `#include` statements. All files that have `.h` extensions are officially called header files.

- Function Declaration

Function declaration (also referred to as function prototype) provide function interface to the compiler.

The interface of a function consists of following information

1. Return type of function
2. Name of function
3. The number and types of items to be passed to the function.

Return type of function identifies the type of data that a function returns to its caller. For example, a function can return to its caller a value of type `int`. In that case, `int` is the return type of the function.

A function that does not return any data to its caller has `void` as its return type.

The name of function is any meaningful identifier as long as it not a reserved word.

The syntax of a function prototype is:

```
return_type_of_function name_of_function(list_of_parameters);
```

Note that the function prototype end with a semi-colon.

For example: `int find_area(int length, int width);`

Function prototype can be written with parameter names and without parameter names

For example: Without parameter names: `int find_area(int, int);` With parameter names: `int find_area(int length, int width);` For a function that do not have parameters, void keyword is used in place of parameter list For example: `void find_area(void);` A function prototype let the compiler get to know the types of parameters, the number of parameters and order of parameters that a function will use. This information enables it to detect type mismatch. Type mismatch is a situation whereby a function is passed values that do not match with the type, the number order of the values that a function expect. For a complex program, it is advisable to have all function prototypes in separate header files. However, for simple programs function prototypes can be included within the body of the program

- `main()` method

The remainder of the program is made of definition of the `main()` function. Function definition has two pieces of information: a header and a body. The header provides the function return type, the function name, followed by parentheses.

return type is the type of the value a function returns to its caller. The `main()` function has `int` as its return type. The `int` value that `main()` function returns to the operation system indicates whether the program has executed successfully or not. It can return either zero(0) or one(1). Zero indicate successful execution of the program and one indicate unsuccessful execution of the program.

The `main()` method is written in two forms:

- Form 1

```
int main(void) {  
...  
return 0;  
}
```

- Form 2

```
int main(int argc, char *argv) {  
...  
return 0  
;
```

The function body contains the statements to be executed when the function is called. These statements are enclosed in braces ({ and }). In the case of the `main()`

function, it has two statements to be executed when it is called. `cout<<"Hello world"<<endl`. The first statement print "Hello world" on the screen. The last statement instruct `main()` function to return an integer (0) to the operating system.

- **Function Definitions**

Function definition has the following properties:

1. return type of function: this is the type of the value that the function returns to the caller. If the function does not return any value to the caller, its return is void
2. function name: this is the name of the function
3. list of parameters: this is a list of the type of parameters, the number of parameters and the order of parameters that the function expect from the caller
4. function body: this is a list of the statements that are to be executed when the function is called. The function body is enclosed within opening and closing curly braces ({, }).

Function definition differs from function prototype in the following ways:

- Function definition must have names of parameters but function prototype must not have names of parameters
- Function definition has function body while function prototype does not have function body

The syntax for a function definition is `return_type_of_function name_of_function(list_of_parameters)`
`{ //function body }`

2.3. Testing

The main objective of testing a program is to uncover and remove errors. A successful software test is the one that has enables one to uncover and remove errors in a program. An error is a mistake that a program make when writing program source code. There are different types of errors

- syntac errors(also known as compiler errors): Wrong use of the syntax rules. These errors are easily detected by the compiler. A program that has syntax errors does not compile
- run-time errors: These are errors that are uncovered when the program is running.

- logical errors: These errors are as a result of wrong logic. They are the most difficult errors to uncover.

2.4. Input and Output Operations

In C++, input and output operations are performed using predefined `cout` and `cin` objects. These objects are defined in `iostream.h` standard library. `cout` print strings on the screen while `cin` reads inputs from the keyboard. The general format of `cout` and `cin` calling them is:

- `cin>>memory_location1>>memory_location2>>...;`

`cout<<"string";`

`cout` is an object of type `ostream` that links a program to a standard output device (monitor), while `cin` is object of type `istream` which links a program with standard input device (keyboard).

The `<<` operator insert characters into the `cout` for display while the `>>` operator extract characters from the `cin` object and store them in memory locations. `>>` operator is referred to as extraction operator while `<<` operator is referred to as insertion operator

Revision Questions

EXERCISE 5.  Explain why comments are important in a program

Example . Briefly explain the role of `main()` method

Solution: The `main()` method is the one that enables a program to start running. It is the first method that is called by the operating system □

EXERCISE 6.  Explain the importance of header files

EXERCISE 7.  C++ is case sensitive. Explain the phrase “case sensitive”

EXERCISE 8.  Differentiate between syntax errors and semantics

References and Additional Reading Materials

- David Flanagan, 2005, *Java in a Nutshell* (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, *The Object-Oriented Thought Process*, 3rd Edition, , Addison-Wesley,

- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 3

Classes and Objects

Learning outcomes

Upon completing this topic you should be able to:

- Declare a class
- Differentiate between member variables and member functions
- Define member functions
- Create objects from a class
- Access data members

3.1. Introduction to Classes

Classes are fundamental building blocks of object-oriented programs.

A class binds data and functions together.

The data and functions of a class are its members.

The data members and member functions details are controlled using private, public and protected keywords.

private, public and protected keywords are also referred to as access specifier and are always followed by a colon (:).

The private section of a class has data members and member functions that are available only within the class(except by friend functions).

The public section makes the data members and member functions within it to be available to all class members and to the other classes members and applications.

The protected section has data members and member functions that are available to the class itself and any other derived class.

The beginning of each section is marked by keywords private:, public: and protected: By default data members and member functions are private.

Normally the data members are put in the private section of a class and the member functions are put in the public section.

3.2. Declaring a class

Specifying the data members and member functions of a class is known as declaring a class.

The syntax for defining a class is:

```
class class_name
{
    private :
        data_type member1 ;
        data_type member2 ;
        . . .
        data_type membern
    public :
        method function1 ;
        method function2 ;
        . . .
};
```

Example :

```
class Rectangle
{
    private :
        int length ;
        int width ;
    public :
        void print_dimension ( void );
};
```

The class definition begins with class keyword followed by the name of the class. The name of a class is an identifier and it is not supposed to be a reserved word. The body of a class is enclosed within the opening curly brace ({) and closing curly brace (}). The closing curly brace is terminated by a semi-colon (;). The body of a class consists of data members and member functions.

3.3. Defining Member Functions

There are two ways of defining member functions. a) Inside the class definition b) outside the class definition

- Definition a member function outside a class definition

The members functions declared within a class can be defined outside a class.

The definitions of these functions are just the same as definition of the normal functions(functions that are not members of a class). But unlike normal functions member functions defined outside a class definition are qualified with the name of the class that they belong to. The name of the class and the member function are separated using the scope resolution operator(: :).

The syntax of defining a member function outside a class is:

return-type class-name :: function-name (parameter list) { function body }

For instance, consider the print_dimension function declared in the Rectangle class above. It may be defined as follows: void Rectangle :: print_dimension(void)

```
{  
cout<<"The Length is: "<<length<<"cm\n" <<"The width is : "<<width<<"cm\n"  
<<endl;  
}
```

- Definition a member function inside a class definition

Member functions can be defined within a class definition. Still consider the find_area() function declared in the class Rectangle above.

This function can be defined in the class as shown below.

```
class Rectangle  
{  
private:  
int length;  
int width;  
public:  
void print_dimension(void)  
{  
cout<<"The Length is: "<<length<<"cm\n" <<"The width is : "<<width<<"cm\n"<<endl;  
}
```

};

Note:

3.4. Creating objects from a class

Once a class has been defined, it can be instantiated.

Instantiating a class is creating an object from a class.

The objects that are created are the ones that are used to call member functions of the class.

A class that can be instantiated is referred to as a concrete class.

There are those classes that can not be instantiated.

Such classes have at least one member a pure virtual function. Such classes are referred to as abstract classes.

The syntax for instantiated a class is:

- Form 1

class-name variable-name;

e.g.

Rectangle rect1;

- b) Form 2

class-name variable-name(arguments);

e.g.

Rectangle rect2(4, 5);

The above two statements creates a class variables rect1 and rect2 of type Rectangle. A class variable is an object. In that case rect1 and rect2 are objects of type Rectangle.

3.5. Accessing class members

All private data members are accessible only by class member functions(this restriction does not apply to friend functions).

It is illegal to have a statement for example within the main method that accesses the value stored in data member.

External functions access private data members by calling public members of a class. The syntax for doing this is

object-name. function-name(arguments);

This is what is referred to as message passing. The receiver of the message is object-name. The name of the message is function-name. The information that is passed to the object in order for the object to respond to the message is “arguments”

E.g.

```
rect1.print_dimension();
```

Below is an example of a C++ program that is made of a class

```
/* _____  
_____  
Rectangle.cpp
```

This program demonstrates the use classes in a

C++ program

```
*/  
#include<iostream.h>  
class Rectangle  
{  
private:  
int length;  
int width;  
public:  
Rectangle(int l, int w);  
void print_dimension(void);  
};  
int main(void)  
{ Rectangle rect1(5, 3);  
rect1.print_dimension();  
return 0;  
}  
/* _____  
_____
```

Definition of Rectangle member function

```
*/  
Rectangle :: Rectangle(int l, int w)
```



```
{  
length=l;  
width=w;  
}  
/* _____  
_____
```

Definition of print_dimension member function

```
*/  
void Rectangle :: print_dimension(void)  
{  
cout<<"The Length is: "<<length <<"cm\n" <<"The width is : "<<width <<"cm\n"  
<<endl;  
}
```

If you compile and execute this program, the output is

The Length is: 5cm The width is : 4cm

Revision Questions

EXERCISE 9.  Differentiate Rectangle rect1; and Rectangle rect2()

Example . Differentiate between function declaration and function definition

Solution: Function declaration is only function header while function definition has function header and function body □

EXERCISE 10.  Differentiate data members and member functions

EXERCISE 11.  Explain the use private, protected and public keywords

EXERCISE 12.  Explain the difference between internal and external definition of member functions

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 4

Friend Functions, Static Functions and Constant Functions

Learning outcomes

Upon completing this topic you should be able to:

- Differentiate a friend function, a static function and a constant function
- Use friend functions, static functions and constant functions

4.1. A Friend Function

A friend function is a function that is not a member of a class (also known as external functions) but it can access data members of the class just like members function of that class.

4.1.1. Making an External Function a Friend Function

An external function is made a friend function by preceding its declaration with friend keyword as shown below:

```
class Account

    private :

        . . .

    public :

        friend void compute_tax ( Account );

        . . .

};
```

Note:

- compute_tax() function is not a member function of Account class. The reserved word friend make this function to behavior like a member function of Account class
- it can not be called using an object of Account class. Therefore, it is called just like a normal function.

- Because `compute_tax()` function is not a member of `Account` class, it can access `Account` class data members through the use of `Account` object. Therefore its argument should be an object of type `Account`
- `compute_tax()` can be declared in any section of `Account` class(private, public and protected sections)

The program below demonstrates the use of friend function

```
/*-----  
Example.cpp This program  
demonstrates the use of friend functions  
----- */  
  
#include <iostream.h>  
class Example  
{  
    private :  
        int x;  
        int y;  
    public :  
Example( int x1, int y1)  
{  
    x=x1;  
    y=y1;  
}  
friend void print_value(Example);  
};  
void print_value(Example e)  
{  
cout<<"The values are "<<e.x <<"and "<<e.y<<endl;  
}  
int main(void)  
{  
    Example myExample(10, 30);  
    print_value(myExample);  
}
```



```
The values are 10 and 30
Press any key to continue
```

Table 4.1: Output from the above program

```
        return 0;
    }
```

4.1.2. Making a Member Function a Friend Function

There are member functions of a class that are friend to other classes.

That means these member functions are able to access data members of that classes as if they are member functions of those classes.

Such member functions are made friend to class as shown below

```
class class-name1
{
    private :
        . . .
    public :
        . . .
    friend return-type class-name2::
function-name();
};
```

Example

```
class B
{
    private :
        . . .
    public :
        . . .
    friend int A :: y(B b);
};
```

where

- friend is a reserved word
- int is the return-type of the function y()
- A is the class that y() function is a member of
- :: scope resolution operators
- y() is the function that is being made a friend of B class.
- y() function behavior as if is a class B member function.

4.1.3. Making all Member Functions to be Friends

All the member functions of a class can be made to be friend to another class.

For example:

```
class B {
private:
    . . .
public:
    . . .
    friend class A ;
};
```

where all A class member functions are now friends to class B

4.2. const member function

A const member function is a member function that does not modify data members in a class.

The reserved word const is appended at the end of the function declaration and function definition as shown below

```
// function declaration
void func() const;
// function definitions
void func() const
{
    function body
}
```

4.3. static member function

A static member function is a member function that has been declared with the static keyword.

A static member function can access only a static variable and it is called using the name of class separated with scope resolution operators.

The syntax of declaring and calling a static member function is

a) declaring a class with static member function

```
class class-name
{
    private :
        . . .
    public :
        . . .
    static return-type function-name()
    {
        function body
    }
};
```

b) syntax of calling a static member

```
return-type function-name()  
{  
    class-name:: static function-name();  
}
```

Revision Questions

EXERCISE 13.  Explain why friend functions are passed object of the class that they have been to be a friend of

Example . Differentiate a member function and a friend function

Solution: A member function can access the data members of a class without using an object while a friend function can only use an object to be able to access the data member. □

EXERCISE 14.  Explain any FOUR characteristics of friend functions

EXERCISE 15.  Show how a member function can be made a friend function

EXERCISE 16.  Differentiate static member functions and constant member functions

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 5

Constructors and Destructors

Learning outcomes

Upon completing this topic you should be able to:

- Differentiate constructor and destructor
- List characteristics of constructor and destructor
- use constructors and destructors

5.1. Constructors

A constructor is a member function that is used for creating objects and initializing the states of the objects.

A constructor has the following characteristics:

- Its is the same as the name of the class within which it has been declared
- It is called automatically
- It does not have return type
- It is always declared within the public section of a class
- It is not inherited vi. A constructor can not be made virtual
- The address of the memory location occupied by a constructor can not be referenced

5.2. Declaration of Constructors

A constructor is declared and defined as follows:

```
class Rectangle
{
private:
int length;
int width;
. . .
```

```
public:
Rectangle(int l, int w); //declaration of constructor
...
}; //definition of constructor
Rectangle::Rectangle(int l, int w)
{
length=l;
width=w;
}
```

5.3. Calling Constructors

Since a constructor is always declared within the public section of a class, it is accessible from external functions(these are functions that are not members of the class that has the class).

For example the main method can call the constructor in the Rectangle class above as shown below.

```
Rectangle rect1(6, 5);
```

This statement is a call to Rectangle constructor in a class called Rectangle. The constructor is also passed two integer values, i.e. 6 and 5. The constructor will construct the object and 6 and 5 values will be used as the initial state of the object. The object that will be created will be referenced by rect1 variable. If we now want to pass a message to the object that has been created in Rectangle class, we have to use rect1 to refer to the object as shown below rect1.find_area(); rect1 is our reference to the object created and find_area() is the message that we are passing to our object. rect1 refers to the object created.

5.4. Types of Constructors

There are three types of constructors:

- Default constructors
- Copy constructors
- Parameterized constructors

5.4.1. A Default Constructor

A default constructor is a constructor that is not passed any arguments when it is called. A default constructor can be created automatically by the compiler. For example, the default constructor for Rectangle class above is :

```
class Rectangle
{
private: . . .
public:
Rectangle();
. . .
}; //definition of default constructor
Rectangle::Rectangle()
{
}
```

Note that the body of this constructor is empty. Now if we have a statement shown below within the main method

```
int main(void) {
Rectangle rect1;
. . .
return 0;
}
```

Statement `Rectangle rect1;` in the above main method is a call to the default. In that case if the Rectangle class do not have the default constructor the compiler will create one automatically.

5.4.2. Parameterized constructors

Parameterized constructors are constructors that are passed arguments when they are called. Unlike default constructors, parameterized constructors are not created automatically by the compiler if they are not defined in a class. And these constructors must be defined. For example:

```
class Rectangle
{
private:
int length;
```

```
int width;  
...  
public:  
Rectangle(int l, int w);  
...  
}; //definition of a parameterized constructor Rectangle::Rectangle(int l, int w) {  
length=l; width=w; }
```

There are two ways of passing arguments to parameterized constructors when they are called: by calling the constructor explicitly and by calling the constructor implicitly. For example `Rectangle rect1= Rectangle( 6, 5);` //explicit call `Rectangle rect1(6, 5);` //implicit call Implicit call is the one that is normally used.

5.4.3. Copy constructors

Copy constructors are constructors that used to create and initialize an object using another object. For example `Rectangle rect1(6, 5);` //a call to a parameterized constructor `Rectangle rect2(rect1);` // a call to a copy constructor to create and initialize rect2 object //using rect1 object

5.5. Destructors

A destructor is a member function that is used to free a location that is held by an object for use by another object.

An object is deleted when the flow of control leaves the scope within which it has been created.

A destructor has the following properties

- Its name is the same as the name of the class
- Its name is preceded by a tilde (~) character
- It is not passed arguments and it does not have return type not even void
- It is called implicitly
- A class can have only one destructor
- A destructor can not be overloaded

For example

```
Rectangle::~~Rectangle()  
{  
    cout<<"An object has been destroyed "<<endl;  
}
```

Note that just like other member functions, constructors can have default arguments and they can be overloaded.

Revision Questions

EXERCISE 17. ✍️ Differentiate a constructor and a destructor

Example 📎. List any FOUR characteristics of a destructor

Solution:

- a) It is not passed any arguments
- b) Its name is the same as a class and it is preceded by tilde (~) symbol.
- c) It is called automatically
- d) It does not have return types

□

EXERCISE 18. ✍️ Explain why a constructor is public and has no return type

EXERCISE 19. ✍️ Differentiate a default constructor, a copy constructor and parameterized constructor

EXERCISE 20. ✍️ Explain the difference between the following two statement:

Car myCar(200);

Car yourCar;

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 6

Storage classes

Learning outcomes

Upon completing this topic you should be able to:

- Differentiate between scope and lifetime of an object
- Use register, auto, static and extern keywords
- List limitations of register variables

6.1. Introduction

In C++, just like in C, variables and functions have scope and lifetime.

The scope (also referred to as visibility) of a variable or a function is a portion of a program in which it is accessible while the lifetime of a variable or a function is the duration of time that it exist during the execution of the program. Information about scope and lifetime of variables and functions is provided by preceding their declaration with the following storage class specifiers:

- auto
- register
- static
- extern

Auto, register, static and extern are keywords.

6.2. Auto storage specifier

All local variables are by default auto. That means they need not be declared with the keyword auto. Since

```
{  
auto int x;  
} is equivalent to  
{  
int x;
```


}

All local variables are created and destroyed automatically.

The word auto is derived from their automatic creation and destruction.

All local variables are created when the block within which they have declared is entered and destroyed when the block is exited. Parameters behavior like local variables and therefore they are created and destroyed automatically.

Only local variables are associated with the auto storage class specifier.

Local variables and parameters have local lifetime and they are visible only in the block of code they have been declared.

6.3. The register storage class specifier

All local variables and parameters are stored in computer memory. However, a compiler can be requested to allocate memory to a local variable and a parameter in the system's register. Examples:

```
register int product=1;
for (register int i=1; i<10; i++)
product*=i;
```

It is possible that the register is not available. In that case the variable is treated like a local variable. Register variables are quickly accessed and therefore the program runs faster. Since registers are limited only a few variables that are frequently accessed need to be stored in the registers. Register variables have the same lifetime and visibility as auto variables.

Some restriction on register variables

a). Only local variables and parameters can be declared with register storage class specifier

6.4. The static storage class specifier

Static variables and functions are declared using static specifier, for example:

```
static int count=0;
```

Static variables have global lifetime. This means that they are created when the program start executing and remain in existence during the entire period of execution. But they are visible only within the block in which they have been declared. Unlike auto variables, static variables are not destroyed after the block in which they are declared has been exited. This means that they keep their values when the

block in which they are declared is exited. Variables that have global lifetime are automatically initialization with zero.

Static keyword can also precede declaration of global variables. A global variable is a variables that has been declared outside all the functions in a program.

In C++, functions can not be declared within other functions. Function can not be declared at internal(local) level. All functions are declared at external(global) level and therefore their their declaration can be preceded with static keyword. A static function is only accessible within source file that it has been declared. Functions in the same source file can call static functions but not functions in other source files. Thereafter different source files can have static functions that have the same name.

6.5. The extern storage class specifier

local variables that have been declared with extern keyword are references to variables with the same name and data type that have been declared later in same file or in a different file.

Revision Questions

EXERCISE 21.  Differentiate scope and lifetime of a variable

Example . Explain why scope and lifetime of variables and functions are important

Solution: Scope and lifetime of variables enables a program to have the same functions and variables without risking name collision

□

EXERCISE 22.  Explain why a function can not be declared with auto and register keyword

EXERCISE 23.  Differentiate static and extern storage classes

EXERCISE 24.  Explain the difference between external storage and internal storage.

References and Additional Readings

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

1. .

LESSON 7

Inheritance

Learning outcomes

Upon completing this topic you should be able to:

- Define inheritance and composition
- Implement inheritance and composition
- Differentiate derived class and base class
- Explain three types of inheritance: public, protected and private

7.1. Introduction to inheritance

Inheritance is an “is-a” relationship. For example, a car is a vehicle.

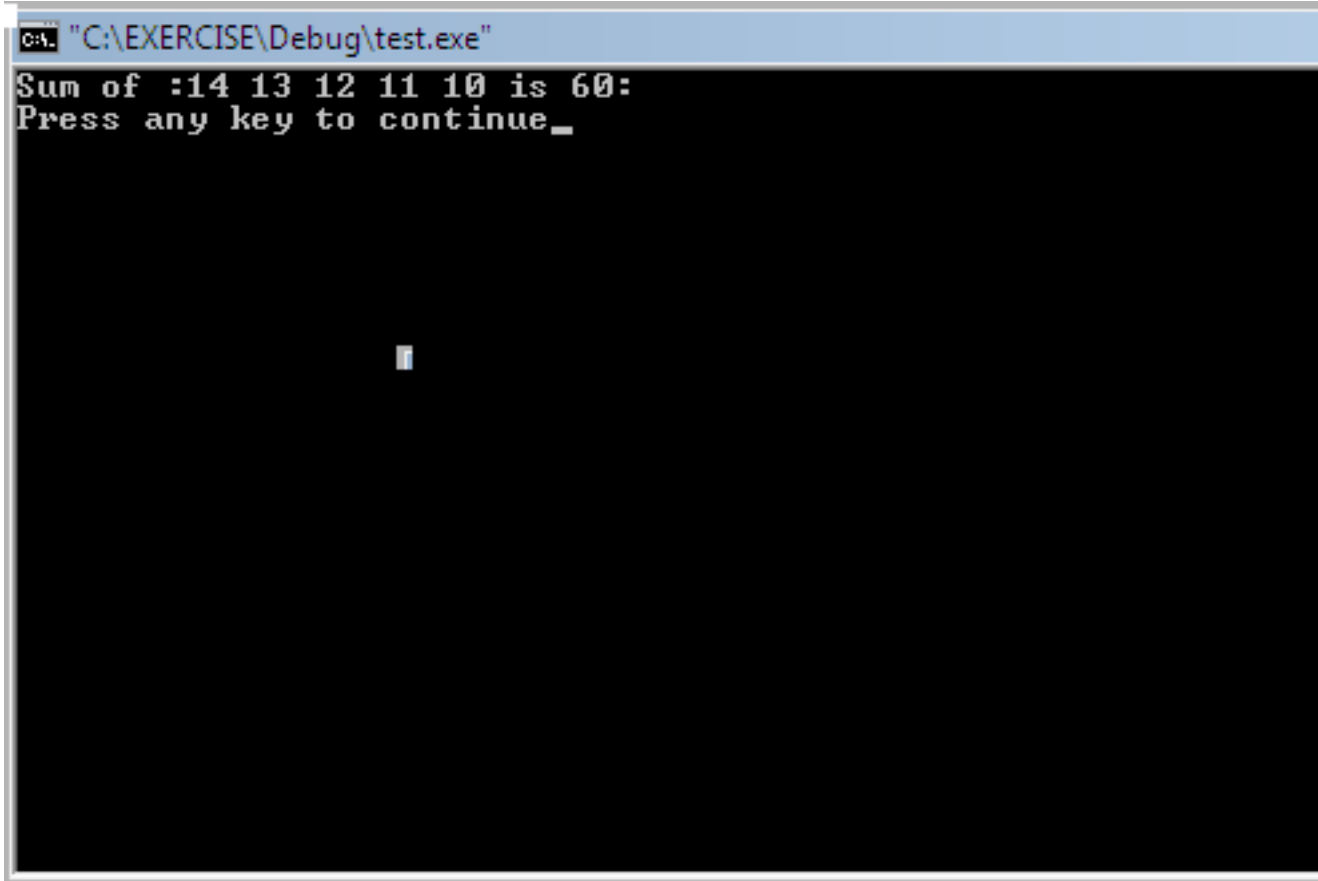
In object-oriented programming, inheritance enables one to create a class from another class. The new class is referred to as a derived class, sub-class or a child class. The existing class is referred to as a base class, parent class or a super class. The derived class inherits properties of the base class. Inheritance can be either a single inheritance or a multiple inheritance. In single inheritance, the derived class has only one base class while in multiple inheritance the derived class has more than one base class.

In the above diagram, Vehicle is the base class. The classes car and lorry are derived from vehicle. Every car, every lorry is a vehicle.

If we have a class Vehicle defined, to specify that class Car is derived from Vehicle class, we use the following syntax:

```
class Car:public Vehicle
{
.
.
.
};
```

Where class and public are keywords. public keyword is a member access specifier. It specifies that all public members of the class Vehicle are derived as public members



```
C:\EXERCISE\Debug\test.exe
Sum of :14 13 12 11 10 is 60:
Press any key to continue_
```

Table 7.1: Inheritance Hierarchy

by the Car class. This means that public members of the class Vehicle become public members of the class Car.

Consider the following definition of Car class

```
class Car: private Vehicle
{
.
.
.
};
```

In this case, the public members of Vehicle class become the private members of the Car class.

The above definition of Car class is equivalent to the definition shown below.

```
class Car: Vehicle
{
.
.
.
};
```

The general format of defining a derived class is:

```
class className: memberAccessSpecifier baseClassName
{
.
.
.
};
```

where memberAccessSpecifier is public, protected or private. When no memberAccessSpecifier is indicated, it is assumed to be private.

Note

private members of base class can not be inherited by the derived class

public members of base class can be inherited by derived class either as public or private members of the derived class

Derived class has additional members

Derived class can redefine public members of the base class

7.2. Redefining and Overloading Member Functions of the Base Class

Redefining a member function is having a member function in the derived class that has the same name and the same set of parameters as another member function in the base class. However, if the base class and derived class have members functions that have the same name but different sets of parameters, then this is function overloading in the derived class.

7.3. Constructors of Derived and Base classes

Constructors initialize data members of a class. When a derived class is defined, it inherits the members of the base class, but it can not access private data members of the base class. When a derived class object is constructed, it must automatically execute one of the constructors of the base class. This means that the constructor of the derived class object must trigger execution of the constructor of the base class object. In that case, a call to the base class constructor is specified in the derived class constructor as shown below.

```
derivedClassName::derivedClassConstructor(parameter):  
baseClassConstructor(arguments) {
```

```
.  
.   
.   
}
```

e.g.

```
derivedClass::derivedClass(int x, int y, char w, int one):  
baseClass(x, y) {  
first=w;  
second=one;  
}
```

7.4. Virtual Functions and Pure Virtual Functions

A virtual function is a function that is bound with its call at run-time(also known as dynamic binding). The compiler does not generate the code to call a specific function. But instead it generate enough information to enable run-time system to generate the code for appropriate function call.

virtual functions are only declared in the base class with the keyword virtual.

Example:

the called class that has been created or derived from another class can have a function that has been inherited from the based class having been redefined. Example

```
class Base{
public:
    virtual void print() {
        cout<<"This is base class"<<endl;
    }
};
```

A pure virtual function behaves just like a virtual function. However, it does not have the body. It is declared in a base class. A class that has a pure virtual function can not be instantiated. Such a class is also referred to as a abstract class. The general format of declaring a pure virtual function is shown below:

virtual return-type function-name(parameters)=0;

example:

virtual void print(void)=0;

A class that is derived from an abstract class must defined pure virtual functions in the base class. Otherwise the derived class will also be an abstract class.

Revision Questions

EXERCISE 25.  Explain the difference between private and protected members of a class

Example . Mark the following statements as true or false

- a) The constructor of the derived class specify a call to the constructor of the base class in the heading of defining derived class constructor
- b) Inheritance is “has a” relation
- c) public members of a base class can be inherited either as public or private by the derived class
- d) When initializing the objects of the derived class, the constructor of the base class is executed last

Solution: a) true b) false c) true d) false

□

EXERCISE 26.  Explain briefly why constructors are not inherited

EXERCISE 27.  Differentiate overloading and redefining member functions

EXERCISE 28.  Explain importance of inheritance

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

1.

LESSON 8

Operator Overloading

Learning outcomes

Upon completing this topic you should be able to:

8.1. Operator Overloading

Operator overloading is a mechanism of making user-defined data types to behavior like built-in types. For example, using in-built data types, line 3 below is legal.

```
int x=10; //line 1
```

```
int y=20; //line 2
```

```
int z=x+y; //line 3
```

However, using user-defined data types, line 3 below can not be understood by the compiler.

```
Rectangle rect1(4, 6); //line 1
```

```
Rectangle rect2(5, 9); //line 2
```

```
Rectangle rect3=rect1+rect2; //line 3
```

Line 3 will be compiled successfully only if plus(+) operator is overloaded. If the plus(+) operator is overloaded we can be able to add two objects as shown below.

```
Rectangle rect1(10);
```

```
Rectangle rect2(30);
```

```
Rectangle rect3=rect1+rect2;
```

8.2. Operation Overloading Function

In order to overload an operator, we must write a function to do that. Such a function is referred to as an operator overloading function. The name of an operator overloading function, is reserved word “operator” followed by the operator to overload. For example, to overload <=(less than or equal to) operator we must write a function with the following name: operator<=

The syntax for declaring an operator overloading function is

```
return-type operator operator-symbol(parameters);
```

Note the following operators can not be overloaded:

a) .

b) .*

c) ?:

d) sizeof

An operator overloading function may be either a member function or a non-member function.

Non-member operator overloading functions are made to be friend function so that they can access data members of other classes.

8.3. Overloading Binary Operators

Binary operators are those operators that requires at least two operands.

For example:

+, -, and *

Let us now look an example of how to overload +(plus) operator

8.3.1. Overloading + operator using a member function

The name of the operator overloading function in this case is: operator+()

Recall operator is a reserved word and it is followed by the name of the operator being overloaded, which is + in this case

Now let us assume we have the following statements:

```
Rectangle rect1(6); //line 1
```

```
Rectangle rect2(7); //line 2
```

```
Rectangle rect3=rect1+rect2; // line 3
```

In line 3, we are adding two objects i.e. rect1 and rect2 and the result is assigned to variable rect3 of type Rectangle.

The two objects are of type Rectangle.

If our operator overloading function is a member of Rectangle class,

line 3 will be interpreted by the compiler as shown:

```
Rectangle rect3=rect1.operator+(rect2);
```

Note that on the right side of the assignment operator(=), we now have a message operator+ being passed to rect1 object and the object is also being passed argument rect2.

Ideally what we have is a call to a function called operator+() that is being passed rect2 value.

And since the function is a member of Rectangle class, we have to call it using an object that has been created from Rectangle class. Note also that operator+ function must return a value which has to be assigned to rect3 variable.

This value is an object of type Rectangle.

Note that the compiler can not interpret line 3 as shown below `Rectangle rect3=rect2.operator+(rect1);`

Let us now write a complete program that as our operator+() function defined.

```
/*-----  
Rectangle.cpp This program demonstrates how binary  
operators are overloaded  
----- */  
  
#include <iostream.h>  
class Rectangle  
{  
private:  
int length;  
public:  
Rectangle(int);  
void print_value(void);  
Rectangle operator+(Rectangle);  
};  
//defining constructor outside  
the class declaration  
Rectangle::Rectangle(int l)  
{ length=l;  
}  
//defining operator overloading  
function outside the class declaration  
Rectangle Rectangle::operator+(Rectangle r)  
{  
Rectangle rect;  
rect.length=length+r.length; return rect;  
}  
void Rectangle :: print_value(void)  
{
```

```
cout<<"The length is :<<length <<endl;
}
int main(void)
{
Rectangle rect1(6);
Rectangle rect2(7);
Rectangle rect3=rect1+rect2;
rect1.print_value();
rect2.print_value();
rect3.print_value();
return 0;
}
```

If you compile and execute this program, it will print the following on the screen

```
The length is 6
The length is 7
The length is 13
```

Note that the syntax of defining an operator overloading function is the same as the syntax of defining other member functions.

8.3.2. Overloading + operator using a non-member function

Let us now see how to overload + operator using a non- member function.

Since the function is a not a member function, it must be a friend function in order for it to access data members.

Consider the three statements below.

```
Rectangle rect1(6); //line 1
Rectangle rect2(7); //line 2
Rectangle rect3=rect1+rect2; // line 3
```

Since we are overload + operator using a non-member function, the compiler will interpret line 3 as shown below `Rectangle rect3=operator+(rect1, rect2);`

Note that now the two objects are passed as arguments to the operator overloading function.

The operator overloading function returns a value of type Rectangle.

Let now write a complete program demonstrating how + operator is overloaded using non member function.

Rectangle.cpp This program demonstrates how binary operators are overloaded

```
*/ #include <iostream.h>
class Rectangle
{
private:
int length;
public:
Rectangle(int);
void print_value(void);
friend Rectangle operator+(Rectangle);
}; //defining constructor outside
the class declaration
Rectangle::Rectangle(int l)
{ length=l;
}
//defining operator overloading function
outside the class declaration
Rectangle operator+(Rectangle r1, Rectangle r2)
{
Rectangle rect;
rect.length=r1.length+r2.length;
return rect;
}
void Rectangle :: print_value(void)
{
cout<<"The length is :<<length <<endl;
}
int main(void)
{
```

```
Rectangle rect1(6);
Rectangle rect2(7);
Rectangle rect3=rect1+rect2;
rect1.print_value();
rect2.print_value();
rect3.print_value();
return 0;
}
```

If you compile and execute this program, it will produce the following output on the screen

```
The length is 6
The length is 7
The length is 13
```

8.4. Overloading Unary Operators

Unary operators are associated with only one operand.

Increment and decrement operators are examples of unary operators.

These operators are overloaded just like binary operators.

The overloading function can be either a member function or a non-member function.

Increment and decrement operators can be either pre-or post decrement and increment operators

8.4.1. Overloading pre-increment/decrement operators using member functions

Consider the statement shown below:

```
Square square1(20); //line 1
Square square2=++square1; //line 2
```

Since we are overloading ++ operator using a member function line 2 will be interpreted by the compiler as follows: Square square2=square1.operator++();

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is message passing. square1 is the object sent the message, and operator++ is the message being passed.

Below is an example of a program that shows how pre-increment operation is overloaded using a member function

```
Rectangle.cpp This program demonstrates how binary
operators are overloaded

/* #include <iostream.h>
class Rectangle
{
private:
int length;
public:
Rectangle(int);
void print_value(void);
Rectangle operator++();
}; //defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{ length=l;
}
//defining operator overloading function
outside the class declaration Rectangle Rectangle
::operator++()
{
length++;
return *this;
}
void Rectangle :: print_value(void)
{
cout<<"The length is :"<<length <<endl;
}
int main(void)
{
```

```
Rectangle rect1(6);
Rectangle rect2=++rect1;
rect1.print_value();
rect2.print_value();
return 0;
}
```

If you compile and execute this program, it will output the following on the screen.

The length is 7

The length is 7

8.5. Overloading pre-increment/decrement operators using a non-member function

Consider the statement shown below:

```
Square square1(20); //line 1
```

```
Square square2=++square1; //line 2
```

Since we are overloading ++ operator using a non-member function line 2 will be interpreted by the compiler as follows: Square square2=operator++(square1);

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is referred to as message passing.

square1 is the object that is passed to the operator++() method.

Below is an example of a program that shows how pre-increment operation is overloaded using a non member function

```
/*-----
Rectangle.cpp This program demonstrates
how binary operators are overloaded
-----*/

#include <iostream.h>
class Rectangle
{
    private:
        int length;
    public:
```

```
        Rectangle(int);
        void print_value(void);
        friend Rectangle operator++();
};
//defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{
    length=l;
}
//defining operator overloading function
outside the class declaration
Rectangle operator++(Rectangle rect)
{
    (rect.length)++;
    return *rect;
}
void Rectangle :: print_value(void)
{
    cout<<"The length is :"<<length <<endl;
}
int main(void)
{
    Rectangle rect1(6);
    Rectangle rect2=++rect1;
    rect1.print_value();
    rect2.print_value();
    return 0;
}
```

If you compile and execute this program, it will output the following on the screen.

The length is 7

The length is 7

8.6. Overloading post-increment/decrement operators using member functions

Consider the statement shown below:

Square square1(20); //line 1

Square square2=square1++; //line 2

Since we are overloading ++ operator using a member function line 2 will be interpreted by the compiler as follows: Square square2=square1.operator++(0);

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is message passing. Zero (0) is a value that is sent to operator++() function.

Zero (0) is an argument that is used to distinguish between pre and post decrement/increment operators.

Zero (0) has no other use.

In fact it is referred to as a dummy parameter.

Below is an example of a program that shows how post-increment operation is overloaded using a member function

/*-----

Rectangle.cpp This program demonstrates
how binary
operators are overloaded

```
*/ #include <iostream.h>
class Rectangle
{
    private:
        int length;
    public:
        Rectangle(int);
        void print_value(void);
        Rectangle operator++(int);
};
//defining constructor outside the class declaration
Rectangle::Rectangle(int l)
{
    length=l;
}
```

```
//defining operator overloading function
outside the class declaration
Rectangle Rectangle :: operator++(int dummy)
{
    Rectangle temp=*this;
    length++;
    return temp;
}
void Rectangle :: print_value(void)
{
    cout<<"The length is :<<length <<endl;
}
int main(void)
{
    Rectangle rect1(6);
    Rectangle rect2=rect1++;
    rect1.print_value();
    rect2.print_value();
    return 0;
}
```

If you compile and execute this program, it will output the following on the screen.

The length is 7

The length is 8

8.7. Overloading post-increment/decrement operators using non-member function

Consider the statement shown below:

Square square1(20); //line 1

Square square2=square1++; //line 2

Since we are overloading ++ operator using a non-member function line 2 will be interpreted by the compiler as follows: Square square2=operator++(square1, 0);

What is on the right side of assignment operator (=) is a call to operator++() function.

In an object-oriented program it is message passing.

square1 and zero (0) are values that are sent to operator++() function. Zero (0) is used to distinguish between pre and post decrement/increment operator overloading.

Zero (0) has no other use.

Below is an example of a program that shows how post-increment operation is overloaded using a non-member function

```
/*-----  
Rectangle.cpp This program demonstrates  
how unary operator  
is overloaded  
-----  
*/ #include <iostream.h>  
class Rectangle  
{  
    private:  
        int length;  
    public:  
        Rectangle(int);  
        void print_value(void);  
        friend Rectangle operator++(Rectangle, int);  
};  
//defining constructor outside the class declaration  
Rectangle::Rectangle(int l)  
{  
    length=l;  
}  
//defining operator overloading function outside  
the class declaration Rectangle operator  
++(Rectangle rect, int dummy)  
{  
    Rectangle temp=*this;  
    (rect.length)++;  
    return temp;  
}
```

```
void Rectangle :: print_value(void)
{
    cout<<"The length is :"<<length <<endl;
}
int main(void)
{
    Rectangle rect1(6);
    Rectangle rect2=rect1++;
    rect1.print_value();
    rect2.print_value();
    return 0;
}
```

If you compile and execute this program, it will output the following on the screen.

The length is 7

The length is 8

8.8. Overloading stream insertion (<<) and extraction (>>) operators

Stream insertion(<<) and extraction(>>) operators are overloaded using non-member functions.

8.8.1. Overloading the stream insertion (<<) operators

Suppose our insertion(<<) operator is being overloaded for Rectangle class.

Consider the following statements:

Rectangle rect; //line 1

cin>>rect; //line 2

cout<<rect; //line 3

Line 2 will be interpreted as follows by the compiler as:

operation>>(cin, rect);

This statement is a call to operator>>() function and the function is passed two arguments: cin and rect.

Line 3 will be interpreted as follows by the compiler as:

operator<<(cout, rect);

This is a call to operator<<() function and the function is passed two arguments: cout and rect.

Note

1. cin and cout are objects of type istream and ostream respective.
2. operators<<() and operator>>() functions return objects of type ostream and istream respectively.

Below is an example of a program, that demonstrates how insertion (<<) and extraction (>>) operators are overloaded

```
/* -----  
overload.cpp This program demonstrates  
how to overload insertion (<<)  
and extraction(>>) operators  
-----  
  
*/  
\#include <iostream.h>  
class Rectangle  
{  
    private:  
        int length;  
    public:  
        Rectangle(int);  
        friend ostream& operator<<  
(ostream& output, Rectangle rect1);  
        friend istream& operator>>  
(istream& input, Rectangle rect2);  
};  
//defining constructor outside the class declaration  
Rectangle::Rectangle(int l)  
{  
    length=l;  
}  
//defining operator overloading  
function outside the class declaration  
ostream& operators<<(ostream& output,  
Rectangle rect1)  
{
```



```
        output<<rect1.length <<endl; return output;
    }
    ostream& operators<<(ostream& input, Rectangle rect2)
    {
        rect2>>rect2.length;
        return input;
    }
    int main(void)
    {
        Rectangle rect1(6);
        cout<<"The value is "<<rect1<<endl;
        cout<<"Enter a value: ";
        cin>>rect1 cout<<"The value is "<<rect1<<endl;
        return 0;
    }
```

If you compile and execute this program, the following will be displayed on the computer screen.

The value is 6 Enter a value: 10 The value is 10

Revision Questions

EXERCISE 29.  Define “friend function”

Example .

Suppose that the operator << is to be overloaded for a user-defined class called mystery. Why must << operator be overloaded using a friend operator overloading function

Solution: The leftmost operand of << is an object of type ostream, and therefore since the class is of type mystery an object of type ostream can not be used to call the operator overloading function. This object has to be passed to the function as an argument. In that case the operator overloading function must be a friend operator overloading function □

EXERCISE 30.  List any FOUR characteristics of a friend function

EXERCISE 31.  Explain the use of “this” keyword

EXERCISE 32.  Consider the following:

Rectangle rect(4,6); //line 1

cout<<rect; //line 2

show how the compiler will interpret line 2

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 9

Template

Learning outcomes

Upon completing this topic you should be able to:

- Define template
- Differentiate between template and overloading
- Declare class template and function template
- Use templates

9.1. Introduction to Templates

Template is the foundation of generic programming. In generic programming, programmers define general classes and functions. template is a family of either classes or functions.

What is a template?

A template is a family of functions and classes. In creation of either a class or a function template, one writes only a single code from where other classes or functions are created.

9.2. Function Template

A function template is a family of functions. For example, if we want to swap integer, characters and floating points, we can overload swap() function as shown below.

```
// to swap integers
int swap(int *a, int *b)
{
    int temp=*a;
    *a=*b;
    *b=temp;
}
// to swap characters
```

```
char swap(char *a, char *b)
{
    char temp=*a;
    *a=*b;
    *b=temp;
}
//to value that have decimal points
double swap(double *a, double *b)
{
    double temp=*a;
    *a=*b;
    *b=temp;
}
```

We have now defined three functions called swap. One function swaps values of type int, the other swaps values of type char while the other swaps values of type double. We have done that by overloading swap() function. However, the three functions swap the values in the same way. In such a case instead of writing these three functions, we could have written a function template and from this template we can create these three function. This is a powerful concepts. In that case we can create a swap function from our template that can be used to swap any data.

The syntax of declaring function template is:

```
template <class Type>
```

```
function-definition
```

Type is an identifier that is used to specify the following:

- a) Type of arguments that will be passed to the function.
- b) The return of the function
- c) To declare variables within the function

For example

```
template <class Type>
Type larger(Type x, Type y)
{
    if (x >= y)
        return x;
```

```
        else
            return y;
    }
```

where template and class are reserved words.

Now we have declared a set of functions that are passed two arguments and return the larger argument.

The types of parameters have not been specified.

This means we can pass to the larger() functions two arguments of the any data types.

Below is a program that shows how function templates are used.

```
/*-----
template.cpp This program demonstrates
how function template is used -----
#include <iostream.h>
//function template declaration
template<class Type>
Type larger(Type x, Type y);

int main(void)
{
    cout<<print the larger value <<larger(20, 30)<<endl;
    cout<<print the larger value <<larger('A', 'B' )<<endl;
    cout<<print the larger value <<larger(3.23, 3.15)<<endl;
    return 0;
}
template<class Type>
Type larger(Type x, Type y)
{
    if(x >= y)
        return x;
    else
        return y;
}
```

If you compile and execute this program it will display the following on the screen:

print the larger value 30

print the larger value B

print the larger value 3.23

9.3. Class Template

Like function template, class template enable one to define a set of code that can be used to create several classes.

The syntax for creating class template is

template<class Type>

class declaration

For example:

```
template <class  Type>
class  Number
{
    private :
        Type x;
    public :
        Number( Type );
        void  print_value ( void );
};
int  main( void )
{
    Number<int>  num1 ( 20 );
    Number<char>num2 ( ' A ' );
    Number<float>num3 ( 3.14 );
    num1. print_value ();
    num2. print_value ();
    num3. print_value ();
    return 0;
}
//defining Number outside the class declaration
template<class  Type>
Numbe<Type>r :: Number( Type  type )
```

```
{  
    x=type ;  
}  
//defining print_value outside the class declaration  
template<class Type>  
void Number<Type>:: print_value(void)  
{  
    cout<<"The value is "<<x<<endl;  
}
```

If you compile and execute this program it will print the following on the screen
The value is 20 The value is A The value is 3.14

Revision Questions

EXERCISE 33.  Suppose cType is a class template and func is a member function of cType. Show the heading of the function definition of func

Example . Suppose cType is a class template, which can take int as a parameter. Explain what the statement below means. cType<int> y;

Solution: it declares y to be an object of the type cType, and the type passed to the class cType is int □

EXERCISE 34.  Find the error(s) in the following code.

```
template<class type>  
class Strange  
{  
    ...  
};  
strange<int> s1;  
strange<type> s2;
```

EXERCISE 35.  Consider the following declaration:

```
template<class type>  
class strange  
{
```


...

private:

type a;

type b;

};

Write a statement that declares `sobj` to be an object of the type `strange` such that the private data members `a` and `b` are of type `int`

EXERCISE 36.  Consider the definition of the following function template

template <class type>

type surprise(type x, type y)

{

return x+y;

}

What is the output of the following statement

a) `cout<<surprise(5, 7)<<endl;`

b)

`string str1="sunny";`

`string str2="day";`

`cout<<surprise(str1, str2)<<endl;`

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

LESSON 10

Files and Streams

Learning Outcomes

Upon completing this topic, you should be able to:

- Define a file and a stream
- Differentiate an input file stream and an output file stream
- Use streams to process data in files

10.1. Introduction

Many programs use large data sets that is stored in secondary memory

The data is stored in files.

There are two basic types of files:

- Text files: The data in the file are characters
- Binary files: The data in the file are in binary form

Files can also be grouped into:

- Sequential files: The data is accessed in the order in which it is stored
- Random files: The data is accessed randomly

10.2. Stream

A stream is a sequence of characters. It is the one that links a program to a file. In object-oriented programs, stream are objects. In C++ streams are objects of type `fstream`. `fstream` is the name of a class that is stored in `fstream.h` file.

An object of type `fstream` is declared as shown below:

```
fstream inputFile;
```

```
fstream outputFile;
```

where:

`fstream` is a keyword

`inputFile` and `outputFile` are names of the streams that are to be linked to files.

Connecting a file with a stream

Connecting a file to a stream is known as opening a file,

- using a constructor function
- using open member function

10.3. opening a file using constructor function.

The syntax of opening a file using constructor is:

```
fstream FstreamName("file_name", opening_mode);
```

where:

`fstream` is a reserved word

`file_name` is the name of the file being opened

`opening_mode` specifies whether a stream is being used for input, output or both

It can be one of the following specifiers:

- `ios::in` open for reading
- `ios::out` open for writing
- `ios::app` append at the end of the file
- `ios::ate` go to the end of the file
- `ios::binary` open as a binary file

For example:

```
fstream outputfile("myfile1.cpp", ios::out); //output file,
```

```
fstream inputfile("myfile2.cpp", ios::in); //input file.
```

Where

`myfile1.cpp` and `myfile2.cpp` are the names of files that are being opened.

`ios::in` and `ios::out` are file open modes for reading and writing respectively.

`myfile1.cpp` is opened for reading and `myfile2.cpp` is opened for writing

10.4. Using open() function

The syntax to open a file using open() function is

```
FstreamName.open("file_name", open_mode);
```

For example.

```
fstream myfile; //line 1
```

```
myfile.open("Data1.cpp", ios::out); //line 2
```

where line 1 is a call to the default constructor in fstream class. fstream is a reserved word. In line 2 open() is a member function of fstream class. It is being called using myfile reference variable. Data1.cpp is the argument that is being passed to the open() function. It is the name of the file that is opened. ios::out is the file open mode. Note that the second argument when opening a file using either a constructor or an open() is file open mode.

A file can be opened in more than one file mode. This is done by combining modes using bitwise OR operator (|). For example file1.open("myFile.cpp", ios::in|ios::binary);

10.5. Checking for successful opening

It is always advisable to check whether a file has been opened successful before carrying out any operation on the file. There are many things that can make opening of a file to fail. For example, a file could be in a different directory or it could not be existing at all. Any attempt to carry out any operation on a file that has not been opened properly will fail.

fail() function is the one that is used to check whether a file has been opened successfully.

fail() function returns true(1) if the opening has failed and false(0) if the opening has succeeded

```
fstream inputfile("myfile.cpp", ios::in);
```

```
if( inputfile.fail())
```

```
//display message if the opening has failed
```

```
}
```

In such a case the program is terminated through the use of exit() function. This function is passed 1 to indicate that the program has not ended successfully.

```
if( inputfile.fail())
```

```
//display message if the opening has failed
```

```
exit(1);
```

```
}
```

10.6. Closing a file

A file that has been opened should be closed after it has been processed. This is done through the use of `close()` function. This function disconnect a stream from a file that it has been connected with.

Example:

```
fstream inputfile("myfile.cpp", ios::in);  
inputfile.close();
```

10.7. Input from and output to a file

Once a file has been connected to an input stream, the input operator `>>` can be used to input data from the file just like the way this operator is used to input data from a keyboard.

Example:

```
fstream inputfile("myfile.cpp", ios::in);  
inputfile>>variable1>>variable2>>...variablen;
```

Similarly, once a file has been connected to an output stream, the output operator `<<` can be used to output data to the file just like the way this operator is used to output data to a keyboard.

Example:

```
fstream outputfile("myfile.cpp", ios::out);  
outputfile<<variable1<<variable2<<...variablen;
```

10.8. Detecting end of a file

It is always advisable to keep on checking whether one has reached the end of a file particularly when the file is an input. This is done using `eof()` function. This function returns `true`(nonzero) value when it has come to the end of a file and zero otherwise.

10.9. File pointers and function for manipulating file pointers

Each file has a get pointer and a put pointer. A get pointer enables a file to be read at certain point where a put pointer enables a file to be written at a certain point.

When a file is opened for reading, the get pointer is automatically positioned at the beginning of the file. Similarly when a file is opened for writing the put pointer is positioned at the beginning of the file. These pointers are moved to any desired positions through the use of `seekg()`, `seekp()`, `tellg()` and `tellp()` functions. `seekg()` and `seekp()` moves the positions of get and put pointers respectively while `tellg()` and `tellp()` returns the current positions of get and put pointers respectively. `seekp()` and `seekg()` are passed two arguments. The first argument is always the number of bytes that the pointer is to be moved, while the second argument is the position where the movement should begin. There are three constants that are used to denote the position where the movement should begin. These constants are: `ios::beg` //this means the movement should start at the beginning of the file `ios::cur` //this means the movement should start at the current position of the pointer `ios::end` //this means the movement should start at the end of the file Note, the movement can be either backward or forward starting at the specified position. This is done by preceding the number of bytes to be moved with either a plus(+)for forward or a minus (-) for backward. For example `inputfile.seekg(0, ios::beg);` //go to the beginning of the file `inputfile.seekg(m, ios::cur);` //move m bytes starting at the current position

Revision Questions

EXERCISE 37.  Define a stream

Example . With aid of examples, explain the difference between opening a file using a constructor and using `open()` function

Solution: `open()` function can be used to opening several files while a constructor can be used to open only one file at a time

□

EXERCISE 38.  Write a program that prompts a user to enter that name of a file. The program should display the content of the file on the screen

EXERCISE 39.  Write a program that counts and print the number of words in a file. The program should prompt a user to input the name of a file

EXERCISE 40.  Explain the various file opening mode

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)

Solutions to Exercises

Exercise 1. a class is a collection of objects that have similar properties while an object is an instance of a class Exercise 1

Exercise 5. comments improve readability/understandability of a program. They enables other programmers to understand the program better Exercise 5

Exercise 9.

Rectangle rect1; This is a call to a default constructor in a class Rectangle

Rectangle rect2(); This is a declaration of a function named rect2() that is not passed any parameters. The function returns a value of type Rectangle.

Exercise 9

Exercise 13.

The object enables friend function to access the data members of the class.

Exercise 13

Exercise 17. A constructor creates objects while a destructor destroy objects

Exercise 17

Exercise 21. Scope of a variable is the portion of the program within which a variable is accessible while life-time of a variable is the duration of time that a variable exist in the program

Exercise 21

Exercise 25. private members of a class can not be inherited by a derived class while protected members can be inherited by a derived class

Exercise 25

Exercise 29. A friend function is a function that is not a member of a class but it has access to the data member of the class

Exercise 29

Exercise 33. `template<class Type> functionReturnType cType<Type>:: func(parameters);`

Exercise 33

Exercise 37. A stream is a sequence of characters. It is a link between a file and a program. In object-oriented programs, stream are objects that connect programs with files. They are the ones that enables programs to read from or write to a file

Exercise 37